

Signals

Part I

Sistemas de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Signals

- **A signal is a notification of an event sent to a specific process**
 - Process receives an integer that maps to a specific event
 - They are a limited form of inter-process communication

- **Notifications are asynchronous**
 - A process may receive a notification at any time, during its execution
 - Signals break the normal sequential control flow of a program

- **If you can't predict when an event occurs, then it is not practical to handle it in the main body of the program**
 - The solution is to write specialized code to handle the signal and execute it only when necessary

Signal sources in a system

■ Hardware interrupts*

- Triggered by external hardware events

■ Exceptions*

- Generated by the CPU for errors such as invalid memory access or division by zero

■ User Input

- Special key combinations (e.g., CTRL+C)

* Hardware interrupts and CPU exceptions are handled by the kernel, which may translate some of them into signals delivered to user processes

Signal sources in a system

■ User-raised signals

- Sent manually using commands like `kill()`

■ Inter-process or OS signals

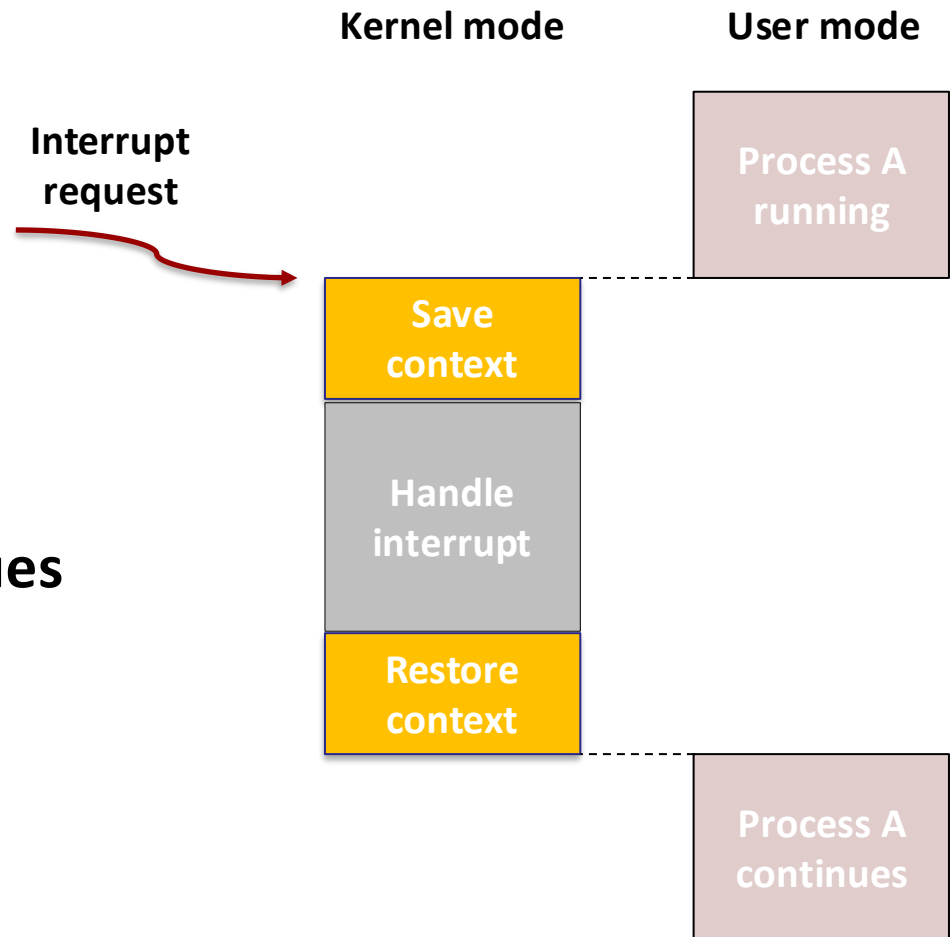
- Sent by other processes or the operating system

■ Self-raised signals

- A process can send signals to itself using functions like `raise()` or `abort()`

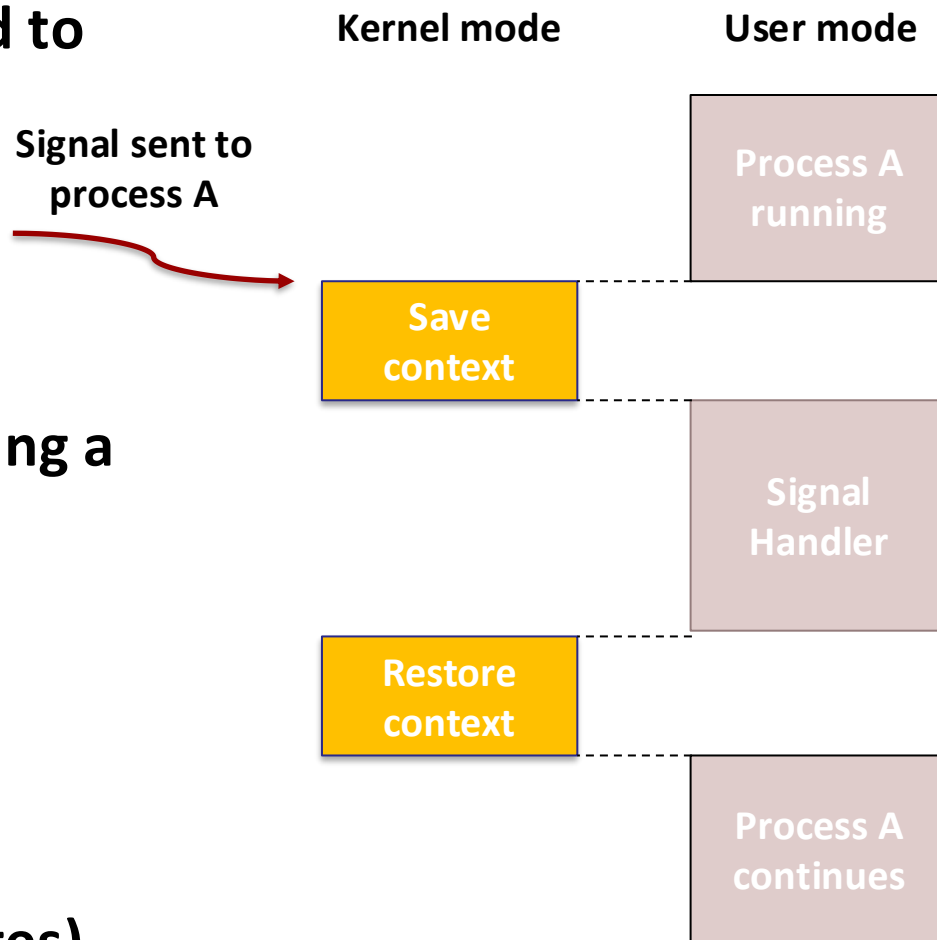
Interrupt handling

- CPU receives an interrupt
- Kernel takes over
 - Saves current context
 - Handles interrupt
 - Restores saved context
- Interrupted process continues executing



Signal handling

- Kernel receives a signal issued to process
- Kernel saves current context
- Process handles signal by calling a specific function
- Kernel restores saved context
- Signaled process continues executing (or maybe terminates)



Which events? Which signals?

- You can list the signals in your system, with the following command:

```
$ kill -l
```

- ...and look at the manual pages:

```
$ man signal
```

- For a complete description of signal handing, see:
 - https://www.gnu.org/software/libc/manual/html_node/Signal-Handling.html

Some examples of signals

Signal	Default action	Description
SIGINT	Terminate process	Interrupt program
SIGILL	Create core image	Illegal instruction
SIGKILL	Terminate process	Kill program (can't be ignored or handled)
SIGSEGV	Create core image	Segmentation violation (illegal memory access)
SIGALRM	Terminate process	Real-time timer expired
SIGSTOP	Stop process	Stop (can't be ignored or handled)
SIGCONT	Discard signal	Continue after stop
SIGCHLD	Discard signal	Child status has changed
SIGUSR1	Terminate process	User defined signal 1
SIGUSR2	Terminate process	User defined signal 2

Dealing with an incoming signal

■ Default action for a specific signal

- Terminate – `SIGKILL`, `SIGINT`, `SIGUSR1`, `SIGUSR2`
- Create core image and terminate – `SIGILL`, `SIGSEGV`, `SIGFPE`, `SIGABORT`
- Stop/Continue – `SIGSTOP`, `SIGCONT`
- Ignore – `SIGCHLD`^{*}, `SIGIO`

■ Programmer-defined action (if signal allows)

- Ignore
- Execute a handler function

^{*} Default action is ignore, but the parent must still call `wait()`/`waitpid()` to reap children

Signaling from the command line

- Use the `kill` command, specifying the signal and the target process

- Some examples:

- Send the default `SIGTERM` to process with PID 1234

```
$ kill 1234
```

- Send `SIGKILL` to process with PID 1234

```
$ kill -SIGKILL 1234
```

- Send `SIGKILL` to process with PID 1234 using the signal's number

```
$ kill -9 1234
```

Signaling using the `kill` function

```
#include <signal.h>

int kill(pid_t pid, int sig);
```

- Sends the signal specified by `sig` to the process specified by `pid`
 - A non-root process cannot send signals to processes owned by other users
- If `pid = 0`
 - `sig` is sent to every process in the process group of the calling process
- If `pid = -1`
 - `sig` is sent to every process for which the calling process has permission to send signals

Example – using the `kill` function

- The child process does not have a signal handler defined for **SIGUSR1**
 - Why does it terminate instead of simply ignoring the signal?

```
int main(){
    pid_t pid;

    pid = fork();

    if(pid > 0) {
        printf("Hello child!\n");
        sleep(5);
        printf("Goodbye child!\n");
        kill(pid, SIGUSR1);
        wait(NULL);
    }
    else {
        printf("Child is running in an infinite loop...\n");
        for(;;);
    }
}
```

Waiting for a signal

```
#include <signal.h>

int pause();
```

- **Causes the calling process to pause until a signal is received**
 - That either terminates the process or causes the invocation of a signal-handling function
- **Upon termination of a signal handler started during a pause, the `pause ( )` call will return**

Example – using the pause function

- It is worth noting that `pause()` only returns if a signal is caught and the handler returns
 - In this specific code, since there is no handler, the signal kills the process and `pause()` never actually “finishes” its execution in the child

```
int main(){
    pid_t pid;

    pid = fork();

    if(pid > 0) {
        printf("Hello child!\n");
        sleep(5);
        printf("Goodbye child!\n");
        kill(pid, SIGUSR1);
        wait(NULL);
    }
    else {
        printf("Child is waiting for a signal...\n");
        pause();
    }
}
```

Important note

■ Standard signals **are not queued**

- If a signal arrives before `pause()` is called, it is lost and will not wake up the process
- `pause()` will not return until another signal arrives

■ This can lead to synchronization issues if `pause()` is used without ensuring the signal is sent at the right moment

■ This is why `pause()` is rarely used in real programs

- `sigsuspend()` is preferred, since it atomically replaces the signal mask of the calling process and suspends the process
- We use `pause()` here for conceptual simplicity to model the basic “sleep-until-signal” mechanism

Signaling using the `raise` function

```
#include <signal.h>

int raise(int sig);
```

- **Sends the signal specified by `sig` to the calling process**
 - It is equivalent to `kill (getpid(), sig)`
- **If the signal causes a handler to be called, `raise ()` will return only after the signal handler has returned**

Signaling using the `alarm` function

```
#include <signal.h>

unsigned int alarm(unsigned int seconds);
```

- Sends the `SIGALRM` signal to the current process in `seconds`
 - If `seconds` is zero, any pending alarm is canceled
- Alarms created by `alarm()` are inherited by children created via `fork()` but are not preserved across `exec()`

Setting the action for a signal

- **By default, each signal has a predefined action**
 - Terminate, stop, continue, ignore, or create a core dump
- **In many programs, the default behavior is not sufficient**
 - We may want to change how the process reacts while a signal is being handled
 - A process's **disposition of a signal** is what it does in response to it
- **To customize how a process responds to a signal, the OS must be informed of the desired action**
 - This is done using the `sigaction()` system call

Setting the action for a signal

```
#include <signal.h>

int sigaction(int sig,
              const struct sigaction *act,
              struct sigaction *oact);
```

- **Modifies the behavior of a process when receiving a specific signal `sig`**
 - `act` specifies the action details for the signal `sig`
 - `oact` (if not `NULL`) stores the previously set action details
- **Let's explore `struct sigaction`**
 - Allows custom handlers, signal masks, and fine-grained control over signal behavior

struct sigaction

- May differ slightly across architectures but these fields must comply with POSIX
 - `sa_handler`
 - Pointer to an ANSI C handler function
 - `sa_sigaction`
 - Pointer to a POSIX handler function (**next class**)
 - `sa_mask`
 - Specifies a set of signals to be blocked while the handler runs (**next class**)
 - `sa_flags`
 - Specifies various flags which can affect the behavior of the signal
- 
- Use one **or** the other

The `sa_flags` field

- **The `sa_flags` field controls important aspects of signal handling, including:**
 - `SA_SIGINFO` – Enables extended signal information to be passed to the handler (used with `sa_sigaction` instead of `sa_handler`)
 - `SA_RESTART` – Automatically restarts certain system calls if they are interrupted by a signal
 - Without this flag, interrupted system calls may fail with `EINTR`
- **Each signal can have its own configuration flags**
 - For the complete set of options, consult the manual pages (`man sigaction`)

Defining signal handlers

- **A signal handler is just a function that you compile together with the rest of the program**
 - Instead of directly invoking the function, you use `sigaction()` to tell the OS to call it when a signal arrives
- **You need to take special care in writing handler functions because they can be called asynchronously**
 - That is, a handler might be called at any point in the program, unpredictably
 - If two signals arrive during a very short interval, one handler can run within another
 - (more on this in the next class)

The `sa_handler` field

■ Used to set an ANSI C handler function

- Provides minimal control over the behavior of signals during their handling

■ There are three possible responses to a signal reception:

- `SIG_DFL` – The default action
- `SIG_IGN` – Ignore the signal
 - Recall that `SIGKILL` and `SIGSTOP` cannot be ignored!
- `void (*sa_handler) (int)`
 - Pointer to an ANSI C signal handler function that is executed upon signal reception
 - This function receives the **signal number as its only argument**

Example – ignoring SIGUSR1

■ This code successfully ignores SIGUSR1

- Could you use the exact same `sigaction()` logic to ignore SIGKILL or SIGSTOP? Why or why not?

```
/* (...) */  
  
int main(int argc, char *argv[]){  
    struct sigaction act;  
  
    /* Zeroes the sigaction structure */  
    memset(&act, 0, sizeof(struct sigaction));  
  
    act.sa_handler = SIG_IGN;  
    sigaction(SIGUSR1, &act, NULL);  
  
    /* SIGUSR1 will now be ignored. */  
    /* (...) */  
}
```


Example – capturing SIGUSR1

■ Async-signal-safety is an important concept

- Why use `write()` instead of `printf()` inside the signal handler? (a topic for the next class)

```
void handle_USR1(int signo){
    /* Inside signal handlers, only async-signal-safe functions should be used */
    write(STDOUT_FILENO, "\nCatch USR1!\n", 13);
}

int main(int argc, char *argv[]){
    struct sigaction act;

    /* Zeroes the sigaction structure */
    memset(&act, 0, sizeof(struct sigaction));

    act.sa_handler = handle_USR1;
    act.sa_flags = SA_RESTART;
    sigaction(SIGUSR1, &act, NULL);

    /* SIGUSR1 will now be captured and handled (ANSI C handler) */
    /* (...) */
}
```

Example – setting an alarm

```
void handle_ALARM(int signo){
    /* Inside signal handlers, only async-signal-safe functions should be used */
    write(STDOUT_FILENO, "\nTime for a coffee!\n", 20);
}

int main(){
    struct sigaction act;

    /* Zeroes the sigaction structure */
    memset(&act, 0, sizeof(struct sigaction));

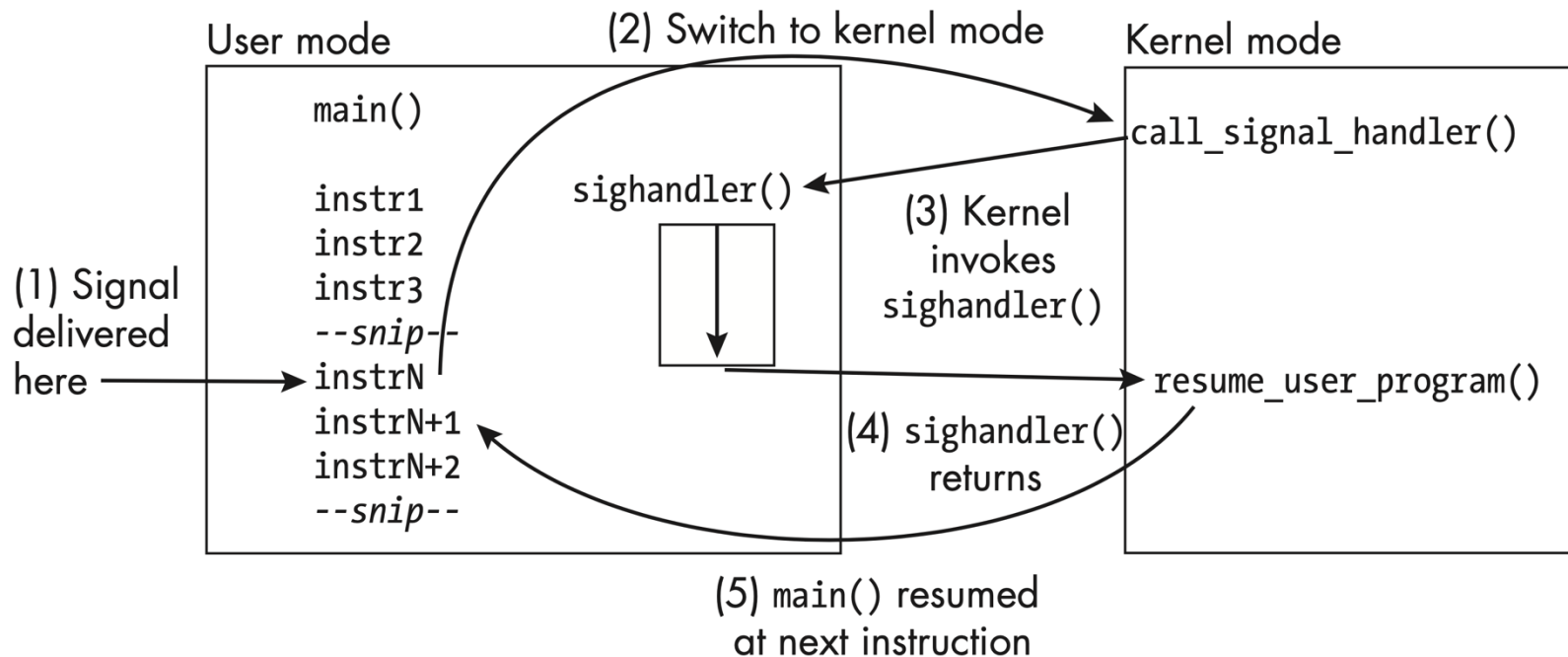
    act.sa_handler = handle_ALARM;
    act.sa_flags = SA_RESTART;
    sigaction(SIGALRM, &act, NULL);

    /* SIGALRM is now set to be catch and handled */

    alarm(10);

    /* You have 10 seconds before interrupting the following code for a coffee */
    do_some_very_long_task();
}
```

Takeaway: capturing a signal



Exercise 1

■ Child process:

- Sleeps for 2 seconds and then sends `SIGUSR1` to the parent
- Enters an infinite loop

■ Parent process:

- Sets up a handler for `SIGUSR1`
- Forks a child process and waits for a signal using `pause()`
- Handles the signal by printing a message
- Sleeps for 2 seconds and sends a `SIGINT` to the child
- Waits for the child to terminate using `waitpid()`